Mechanistic Interpretability for End-to-End Self-Driving

Manav Gagvani[1], **Benjamin Namikas**[2], **Sivamurugan Velmurugan**[2], **Nysa Kumar**[2]
Shrey Sharma[2], **Thomas Peterson**[2], **Pratyush Mathur**[2], **Hyunseo Chang**[2], **Rohan Paleja**[2]

[1]Department of Electrical & Computer Engineering, Purdue University
[2]Department of Computer Science, Purdue University
{mgagvani, bnamikas, svelmuru, kumar753,
shar1159, peter922, mathur91, chang899, rpaleja}@purdue.edu

Abstract: End-to-end (E2E) autonomous driving models achieve strong planning performance but remain largely opaque, limiting observability and trust. We apply mechanistic interpretability to E2E driving by training Sparse Autoencoders (SAEs) on the internal activations of a simple, single-camera end-to-end path planning model. Unsupervised clustering of SAE latents reveals interpretable structure aligned with route intent, vehicle speed, and braking behavior, recovering route intent with 98.3% accuracy without supervision. Through paired counterfactual image edits, we further show that specific SAE features are visually grounded and systematically respond to scene changes such as traffic light state and stop sign presence. Steering individual SAE latents produces predictable, monotonic changes in planner outputs including lateral displacement, curvature, and braking. Finally, we demonstrate a practical application of these representations: we build a lightweight adapter trained on SAE latents that reduces trajectory selection error by 3.44%, outperforming an equivalent adapter that omits SAE latents, confirming that the sparse representation captures useful information for interpretability.

Keywords: End-to-end driving, Mechanistic interpretability

1 Introduction

Road safety remains one of the most urgent motivations for autonomous driving research. Each year, road traffic kills approximately 1.2 million people worldwide, making autonomous vehicles both a research and public health problem [1, 2]. Companies such as Wayve are deploying learned end-to-end (E2E) autonomy models on public streets [3, 4], but as E2E autonomous driving systems transition from simulation to deployment in the real world, post-deployment safety through observability and understanding of model behavior becomes increasingly important.

Over the past decade, the field of autonomous driving (AD) has shifted from modular pipelines with handcrafted perception, planning, and control modules, to end-to-end models that directly learn from data to map raw sensor data and ego state into driving decisions [5, 6]. Despite advancements in the application of Vision-Language-Action (VLA) models for autonomous driving [7, 8, 9], the black-box nature of E2E models diminishes trust and restricts practical applications due to limited safety bounds and understanding of model behavior in diverse or out-of-distribution scenarios. To address this shortcoming, some E2E methods produce intermediate outputs such as bird's-eye-view occupancy or motion prediction [10, 11], providing clues as to a model's understanding of a scene but little insight within the exact inner workings of the network.

To better understand E2E models, we leverage techniques from mechanistic interpretability [12, 13]. Specifically, we apply Sparse Autoencoders (SAEs), an unsupervised learning technique that disentangles polysemantic representations by projecting them into a sparse latent space. This technique

has been used to find highly interpretable features in LLMs [14] and visuomotor VLA policies [15, 16]. To our knowledge, we are the first to apply these techniques to E2E driving policies.

Our contributions are as follows:

1. An elegant, high-performance end-to-end planning module using trajectory scoring, without any intermediate labeled representations or large language modeling backbones, that achieves competitive open-loop planning performance on the Waymo Open Dataset's E2E driving benchmark [17].
2. We show that trained SAE features organize into interpretable clusters aligned with route intent and kinematic behavior, and that these features respond systematically to counterfactual visual changes in driving scenes.
3. We show that SAE latents are useful representations for model steering and improvement. Latent steering produces structured changes in planner behavior consistent with clustering and image edit analyses, and using SAE latents as inputs to an adapter subnetwork improves model performance over the original dense query activation.

2 Related Work

Our work combines three areas of research: end-to-end autonomous driving, interpretability for driving policies, and mechanistic interpretability through sparse feature learning.

End-to-End Autonomous Driving – Since the seminal paper, "End to End Learning for Self-Driving Cars" [18], which trained a small CNN to map front-facing camera images to steering commands, there has been an explosion of model architectures for the task of end-to-end autonomous driving. Two major categories of trajectory generation methods are direct trajectory regression and trajectory scoring [5, 19, 20]. One key baseline model is TransFuser [21, 22, 23] which fuses multiple cameras and LiDAR point cloud inputs by using a cross-attention feature pyramid architecture. Regressing a single trajectory limits the multimodality of the planner's output, leading to the development of trajectory scoring methods [24]. This category can be further split into networks which solely use a pre-computed vocabulary of actions [25, 26, 27], and methods which dynamically generate trajectory proposals to be selected from [28, 29, 30]. Our model falls under the class of trajectory scoring approaches, specifically methods that dynamically generate trajectory proposals and then select among them using a learned scoring head. Our model is unique in that it does not incorporate intermediate labeled representations such as HD maps or bird's-eye-view occupancy, and only uses a single camera as visual input.

Interpretability in End-to-End Autonomous Driving – Existing E2E models approach interpretability through auxiliary output heads or textual reasoning, but do not identify internal features used by the planner to generate or select trajectories [31, 11]. Existing work exposes intermediate outputs such as motion prediction and predicted scene occupancy [10, 11], helping understand what the model's internal representations can extract from given inputs. More recently, vision-language models and VLA models have been used to improve the transparency and interpretability of E2E autonomous driving models [7]. For instance, DriveLM and the Alpamayo-R1 VLA use large backbones to perform visual question answering [8, 31]. Hint-AD inserts a pretrained vision-language model into existing E2E architectures, producing reasoning and 3D dense captioning which are aligned to the model's internal representation [9]. These existing methods improve interpretability by exposing additional model outputs, such as occupancy, motion prediction, captions, or language-based reasoning. In contrast, our work focuses on the planner's internal representations: we study whether the internal features used to produce trajectories can be identified and linked to driving behavior.

Mechanistic Interpretability and Sparse Autoencoders – Mechanistic interpretability studies internal model representations rather than input-output behavior alone. A key obstacle is *polysemanticity*: individual neurons often respond to multiple unrelated concepts, likely because models store features in superposition [32, 33]. Sparse autoencoders (SAEs) address this by learning a dictio-

nary that reconstructs hidden activations with sparse coefficients, yielding features that are more monosemantic and interpretable than raw neurons [14]. Recent work replaces L_1 sparsity penalties with TopK activations, which directly control the number of active latents, and adds an AuxK loss to reduce dead latents [12]. Critically, Marks et al. [34] demonstrate that sparse feature circuits can be identified and edited to steer model behavior. Most relevant to the E2E autonomous driving setting, Swann et al. [15] extend this framework, training SAEs on VLA robot manipulation policies and demonstrating that individual features can causally steer trajectory behavior. These works show that SAEs can expose interpretable and causally meaningful features in language models and VLA policies. However, it remains unclear whether the same approach can reveal useful internal structure in E2E autonomous driving models, which can lack language modeling backbones. Our work addresses this by applying SAEs to the internal activations of an E2E driving model and evaluating whether the resulting features are aligned with kinematic and visual features, and if those features can alter model behavior.

3 Method

3.1 End-to-End Driving Model

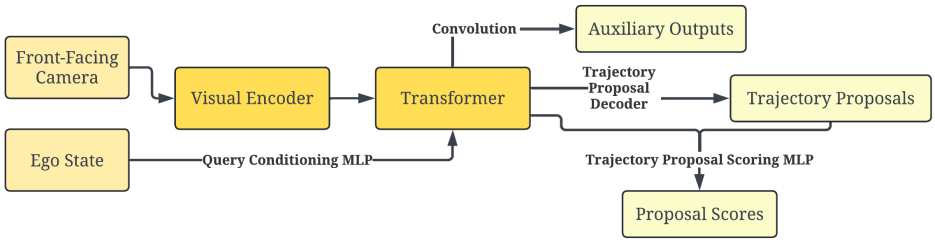


Figure 1: **Model Architecture.** Our Simple Monocular Trajectory Scorer (SMTS) uses only the front camera, past ego state, and route intent as input. Past state and intent initialize a query token, which attends to adapted visual tokens through four transformer blocks. The final query predicts candidate control sequences, rolls them out into future trajectories using a kinematic motion model, and scores each trajectory with a learned proposal scorer. A supervised depth head is used as an auxiliary training output.

We develop an elegant, high-performance E2E architecture, the Simple Monocular Trajectory Scorer (SMTS) shown in Figure 1. Many existing methods either use large VLM/VLA modules or incorporate diverse sensor inputs such as LiDAR or multiple cameras. To isolate the internal planning representation, we use a monocular E2E planning model which consists of four stages: visual encoding, query conditioning, trajectory proposal decoding, and trajectory proposal scoring.

Visual encoding – Building on transformer backbones used in Kirby et al. [28], the model first extracts patch-level features from the front-facing camera image using a vision transformer (ViT) backbone. We freeze the backbone during training and learn a lightweight convolutional visual adapter on top of its feature map, which is then flattened into N_t visual tokens each of dimension d. An auxiliary dense depth head encourages the adapter to preserve geometric information useful for planning.

Query conditioning – Li et al. [35] and Zhai et al. [36] showed that past ego status alone can achieve high performance on open-loop autonomous driving benchmarks, creating a risk that the model learns shortcuts that bypass visual scene understanding. To make this information available while keeping it localized, we inject past ego motion and route intent only through the initial query. Concretely, route intent is converted to a one-hot vector and concatenated with the past ego state to produce a vector v. A learned linear projection maps v to an initial query $Q \in \mathbb{R}^d$. This query is the only token updated by the transformer decoder, so the decoder must fuse v with visual tokens through attention rather than directly emitting trajectories conditioned on the ego state.

Trajectory proposal decoding – Before decoding proposed trajectories, we add a learned positional encoding to each token. Next, a transformer decoder, which is a stack of 4 transformer blocks, produces the final query Q'. An MLP transforms Q' to a set of N_p trajectory proposals, each of which is a sequence of T poses predicted from the current timestep t up to a future horizon at $t+T$. Each timestep consists of $(\dot{v}, \omega) \in \mathbb{R}^2$, which are rolled out into $(x, y) \in \mathbb{R}^2$ future poses via a kinematic model in the ego agent's local reference frame at timestep t.

Trajectory proposal scoring – The scoring head assigns a scalar cost to each decoded proposal rather than producing a single trajectory directly. Each proposed (x, y) trajectory is first flattened and embedded with an MLP. The resulting trajectory embedding is concatenated with the detached final planner query $\text{Detach}(Q')$. A second MLP maps this concatenated representation to one scalar score per proposal. At inference, the model selects the proposal with the lowest score, yielding the predicted best trajectory.

3.2 Sparse Autoencoder with AuxK Loss

To analyze the internal planning representation of SMTS, we train sparse autoencoders using the TopK formulation and AuxK auxiliary loss, following prior work on VLA interpretability [12, 15]. We apply the SAE to the query activations of our compact end-to-end planning model. Since the transformer query concentrates route intent, past ego motion, and visual evidence before proposal generation and scoring, it provides a compact target for mechanistic analysis.

For each transformer block $\ell \in \{0, 1, 2, 3\}$, we extract the query activation after the block $x_\ell \in \mathbb{R}^d$ and train a separate SAE for that block. Each SAE first normalizes activations by subtracting the training-set geometric median, removing the per-sample feature mean, and applying L_2 normalization. The normalized activation $\tilde{x}$ is encoded with a TopK sparse nonnegative code, producing the latent z in Equation (1).

$$z = \text{ReLU}\left(\text{TopK}(W_{\text{enc}}\tilde{x}, k)\right). \tag{1}$$

The decoder reconstructs the normalized activation $\hat{x}$ as $W_{\text{dec}}z$. Finally, the training objective is $\mathcal{L}_{\text{rec}} + \alpha_{\text{aux}}\mathcal{L}_{\text{aux}}$, where $\mathcal{L}_{\text{rec}}$ is normalized reconstruction error, $\mathcal{L}_{\text{aux}}$ is the AuxK residual loss over inactive or dead latents, and α is a training hyperparameter. The resulting sparse latents are used as the feature space for clustering, counterfactual visual edit analysis, latent steering, and scorer correction.

3.3 SAE Analysis and Model Steering

After training SAEs on SMTS query activations, we analyze the learned latent space at four levels: whether features organize into coherent groups, whether latents are directly affected by visual input, whether individual features respond to controlled visual changes, and whether intervening on those features causally changes planner behavior. We then use the same latent representation in a lightweight scoring adapter to test whether the sparse features contain information useful for improving trajectory selection.

We first analyze co-activation structure among SAE latents. We perform this analysis on the SAE latent z_3 from the final transformer block because this representation is the planner query immediately before trajectory proposal decoding and scoring.

Using the SAE activations from the final block, z_3, we first filter individual features by activation rate, retaining those with activation frequencies in $[f_1, f_2]$. This filters out latents which are exceedingly rare and latents which activate on most scenes. For the retained features, we compute the pairwise Pearson correlation of binarized SAE activations $\mathbf{1}[z_{ij} > 0]$ over the full validation set. We then construct an undirected graph whose nodes are retained SAE features, with an edge between features a and b when their correlation ρ_{ab} exceeds a threshold τ. Connected components of this graph with at least m features define feature clusters. To interpret a cluster C, we rank scenes in the validation set by activation score $\sum_{j \in C} z_{3,j}(i)$ and inspect the highest-scoring scenes.

These are scenes in which many features in the cluster fire strongly, making them natural candidates for interpreting the shared factor represented by the cluster. To characterize each cluster, we summarize scene, trajectory, scoring, and error statistics over the validation set. We compute z-scores using these statistics, indicating which driving conditions and planner behaviors are overrepresented among scenes selected by the cluster. Using the 50 highest-scoring scenes, we define each cluster's semantic label as the combination of the most common route intent and the sign of the mean speed z-score in that representative subset.

After identifying co-activated feature groups, we next test whether SAE features are grounded in the visual scene rather than only reflecting ego motion or route intent. Since the query is initialized from past ego motion and route intent, clustering structure could be affected by metadata inputs. To disambiguate this, we quantify how latent variation depends on the input image, intent, and ego history by systematically recombining these inputs. For a set of validation samples, we inference the model over the sample's image and six different route intent and ego history combinations. Using the resultant set of latent vectors, we perform an ANOVA variance decomposition, separating the variation in latents into the contributions and interactions between the three model inputs.

To identify SAE features which correspond with items relevant to driving, we construct paired counterfactual image edits by holding all inputs constant while only changing the input image, producing matched pairs where changes in the SAE activations are solely due to the visual change. For each scene, a YOLO object detector provides context to a VLM prompt generator, which produces scene-consistent edit prompts (e.g., only adding pedestrians where a crosswalk is visible). An image editing model applies each prompt, forming matched pairs (I, p, r) and (I', p, r) with identical past state p and intent r, yielding query tokens (h, h') and latents (z, z').

We quantify each edit via token-level shift $\|h' - h\|_2$, sparse latent shift $\|z' - z\|_2$, and signed per-feature delta $z' - z$. We then validate feature monosemanticity by directly steering individual latent features via $z'_i = \max(0, z_i + \alpha s_i)$, where s_i is a per-feature normalization factor, and measuring downstream planner behavior.

For each feature, we select scenes in which it is most active and sweep α over a fixed set of values $\mathcal{A}$, producing a modified latent $\tilde{z}$ and updated final query $\tilde{Q}$, from which new proposals and corresponding scores are computed. At each intervention level $\alpha_j \in \mathcal{A}$, we measure how the intervention changes both the selected trajectory and the proposal distribution. Specifically, we track lateral displacement, curvature, braking and acceleration magnitude, score margin, and proposal spread. These metrics, along with feature quality, are defined in Appendix F.

Finally, we ask whether the SAE latent space is not only interpretable, but also useful for improving the planner's trajectory selection. This latent representation can be used to improve the performance of the scoring submodule after training without requiring supervised fine-tuning of the full model. We propose an auxiliary adapter module to predict the original scoring head error ϵ_{scorer} using SAE latent activations. Because model error largely stems from misclassifying the best generated trajectory, we target the scoring submodule directly. The adapter predicts each proposal's scorer error using an L_1 loss. At inference time, it corrects predicted scores by adding predicted error.

4 Results

We present results demonstrating that 1) SMTS is comparable to existing E2E planners and 2) that extracted SAE features are both **interpretable** and **steerable**: we cluster SAE features to reveal interpretable structure, use counterfactual image edits to show features are visually grounded, demonstrate that individual features can predictably alter planner output, and show that SAE features improve downstream performance via a learned adapter.

4.1 Open-Loop Planning

All experiments use a single front-facing camera as input to a frozen pretrained Perception Encoder (ViT-S) [37], with decoder and SAE hidden dimension 384, trained on 250,000 WOD-E2E [17] samples for 10 epochs with a learning rate of 1×10^{-4}. WOD-E2E mines rare long-tail scenarios

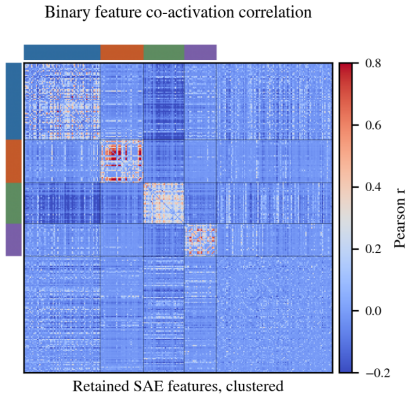
(construction, intersections, multi-lane maneuvers) absent from existing benchmarks [17, 38]. Our SMTS model achieves an ADE of 1.40 m on the validation split, outperforming Latent TransFuser (1.50 m), GTRS-Dense (1.48 m), and DrivoR (1.49 m) [21, 29, 28]. On the private WOD-E2E test evaluator, SMTS also achieves the highest average Rater Feedback Score (7.765 vs. 7.688 for GTRS-Dense and 7.645 for DrivoR), leading on 6 of 11 scene categories. We further show that SMTS scales more efficiently with limited data compared to competing methods, as well as full WOD-E2E results, in Appendix B.

4.2 Unsupervised Feature Clustering

Next, we follow the clustering procedure in Section 3.3 to test whether the learned SAE latents exhibit organized structure. We analyze the SAE latents from the final transformer block, z_3, over the full WOD-E2E validation set. Using pairwise Pearson correlations with $\tau = 0.35$, we construct a co-activation graph by retaining connected components of at least three features, yielding four clusters of 64, 36, 34, and 27 latents (Figure 2). The same four groups are recovered by greedy modularity maximization [39], a threshold-free alternative to connected components, with 94.3% of features receiving the same label (Appendix C.2).

To quantify alignment with semantic driving structure, we assign each scene to the cluster with the highest mean activation $|C_k|^{-1} \sum_{j \in C_k} z_{3,j}(i)$, and map clusters to intents by majority vote. This assigns C_0, C_2 to driving straight, C_1 to left turns, and C_3 to right turns, recovering route intent with 98.3% accuracy overall (per class accuracies are 99.8% for straight, 92.9% for left turns, and 86.4% for right turns).

Among the top 50 scenes ranked by cluster activation, C_0 corresponds to stopped or slow straight-driving scenes, with mean speed 1.04 standard deviations below the validation mean, while C_2 corresponds to high-speed straight-driving scenes, with mean speed 0.98 standard deviations above (Appendix C.2–C.3). Thus, the co-activation structure recovers both the explicit route intent variable and an additional continuous driving factor, speed, without using either signal to construct the clusters. This unsupervised alignment suggests that the SAE has learned an internally organized representation of planning-relevant scene structure.



Cluster-level scene statistics

Cluster	# feat.	Top-50 intent	Interpretation	Speed z	Brake z	Entropy z
C0	64	Straight (100%)	Stopped / slow straight	-1.04	-0.52	+0.03
C1	36	Left (100%)	Stopped left turn	-1.07	+2.00	-1.75
C2	34	Straight (100%)	High-speed straight	+0.98	-0.30	+0.48
C3	27	Right (100%)	Stopped right turn	-1.04	+1.12	-0.94

Figure 2: **Unsupervised SAE feature clusters.** *Left*: Pairwise Pearson correlation matrix of binarized SAE latent activations. Features are ordered by connected components, revealing four major clusters. *Right*: Cluster-level summaries for the connected components obtained at $\tau = 0.35$, including cluster size, dominant route intent, and standardized driving/model statistics for the scenes that most strongly activate each cluster. The clusters separate left- and right-turn behavior from driving straight, while the two straight clusters further separate low-speed and high-speed scenes.

4.3 Input Conditioning Analysis

Since the query is initialized from route intent and past ego motion, we test whether the final transformer block's latent representation remains dependent on visual input when these inputs are controlled. We evaluate 9,000 validation images under a combination of three route intents and two

fixed ego histories, and apply an ANOVA decomposition to the resulting latent activations. Table 1 shows that the image accounts for 14.3% of latent variance directly, and a further 18.5% through interactions with intent and ego history. The image $\times$ intent and image $\times$ ego history terms (7.2%, 6.9%) are significant, since if the latents simply re-encoded these inputs with no contribution from the image, these terms would be zero. The latents therefore combine visual evidence with intent and ego state rather than copying them forward.

Table 1: Share of SAE latent variance explained by each input.

Source	Var. (%)	Source	Var. (%)	Source	Var. (%)
Image	14.3%	Intent $\times$ ego history	16.9%	Image $\times$ intent $\times$ ego history	4.4%
Intent	26.1%	Image $\times$ intent	7.2%		
Ego history	24.0%	Image $\times$ ego history	6.9%		

4.4 Counterfactual visual edits

Next, we show that SAE features are visually grounded, rather than merely reflecting simple kinematic features such as route intent. We use the procedure described in 3.3 to perform this analysis: holding each sample's past ego state and route intent fixed while modifying only the front-facing camera image, and then measuring changes in the output trajectory and SAE latents.

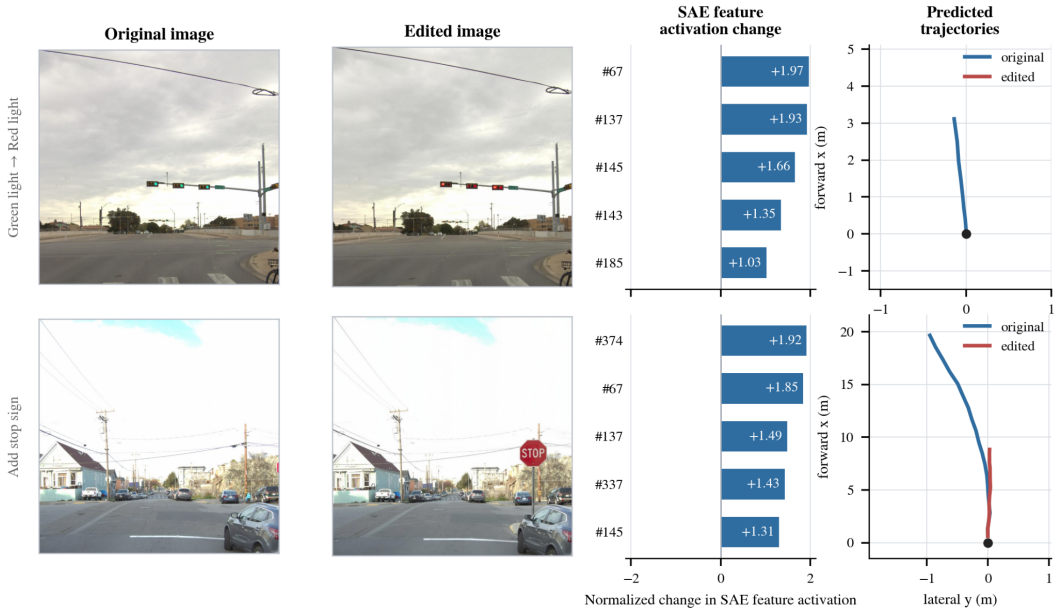


Figure 3: **Counterfactual visual edits reveal visually grounded SAE features.** Each row shows a paired intervention in which the front-facing image is edited while ego history and route intent are held fixed. From left to right, columns show the original image, edited image, largest normalized changes in SAE feature activations, and the selected trajectory before and after the edit. *Top*: changing green traffic lights to red produces large shifts in a small set of SAE features and changes the selected trajectory from forward motion to stopping. *Bottom*: inserting a stop sign reduces the selected trajectory displacement.

Figure 3 shows two representative edits. Each row shows the original image, the edited image, the top 5 SAE feature activation changes (normalized by each feature's standard deviation across the validation set), and the resulting selected trajectory overlaid on the original. In the first edit, switching a green traffic light to red produces a shift in specific SAE features, and changes the predicted trajectory from forward motion to a full stop. In the second edit, inserting a stop sign

into the scene produces a different shift in SAE feature activations, and also decreases the predicted displacement of the vehicle.

These examples demonstrate that SAE features capture semantically meaningful visual changes that affect trajectory selection, motivating our latent steering analysis in Section 4.5.

4.5 Latent Steering

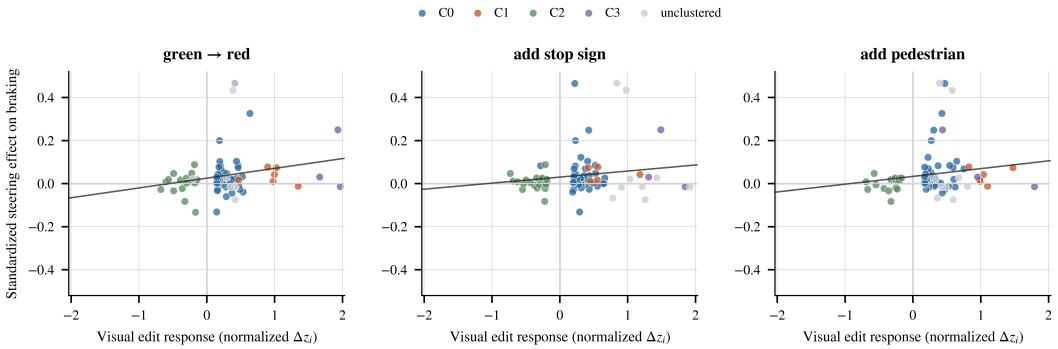


Figure 4: **SAE features have aligned steering effects.** Each panel compares counterfactual visual sensitivity with latent steering. Each point is an SAE feature selected among the most responsive features for that edit. The x-axis shows the feature's normalized signed activation change under the visual edit, while the y-axis shows the standardized change in braking magnitude when the same feature is manually increased through latent steering. Across green-to-red traffic light edits, added stop signs, and added pedestrians, relevant features show a consistent positive alignment between visual response and expected braking behavior.

To connect visual sensitivity with steering behavior, we compare two independently computed quantities for each SAE feature. For an edit direction e, we replay the original and edited images through the planner, encode the resulting query activations with the SAE, and define $x_{e,i}$ as the feature's mean signed activation change across successful edits divided by its validation-set standard deviation, shown in Figure 4.

We compare each feature's visual response to its direct steering effect on braking. For feature i, let y_i be the mean standardized change in braking after amplifying that feature and rerunning the planner. Figure 4 plots y_i against the visual edit activation response $x_{e,i}$ for the most relevant features under each stop-inducing edit. Alignment is consistently positive across green-to-red light edits, added stop signs, and added pedestrians: 61% of interventions match the visual edit sign, with Pearson $r = 0.172$. This indicates that visually responsive SAE features also tend to produce the expected trajectory change when steered directly.

4.6 Lightweight Scorer Adaptation

Finally, we test whether the SAE representation is useful not only for interpretation, but also for improving the planner after training. Rather than fine-tuning the full SMTS, we freeze the base planner and train an adapter on top of its existing proposal-scoring submodule, as described in Section 3.3.

Adding the SAE-based adapter reduces ADE@5s by 3.44% relative to the base model. Equivalently, the mean selected rank improves from 16.7 to 15.6 out of 50 proposal trajectories, where proposals are ranked by ADE to the ground-truth trajectory. Figure 5 shows representative examples where score correction changes the selected trajectory, decreasing planning error.

As a control, we train the same adapter architecture without SAE latents, replacing them with the raw activations from the layer that the SAE is trained to reconstruct. This simplified control recovers 80% of the ADE improvement achieved by the SAE-based correction adapter. In addition, we replace the frozen SAE encoder with a randomly initialized trainable encoder with the same architecture and

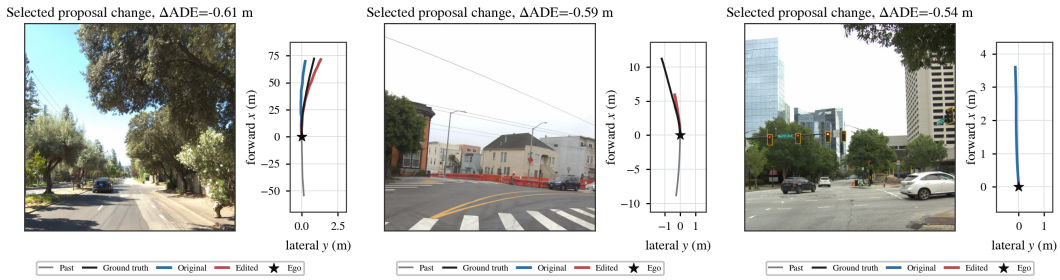


Figure 5: **SAE-based scorer correction improves trajectory selection.** The scoring adapter modifies proposal scores using SAE latents while leaving the base planner and generated proposal set fixed. Each example shows a case where score correction changes the selected trajectory to a lower-error proposal. *Left:* The adapted scorer selects a trajectory that turns further into the side lane rather than continuing straight. *Center:* The adapted scorer selects a higher-speed left-turn trajectory instead of a slower trajectory that cuts across the lane. *Right:* The adapted scorer selects a stationary trajectory at a red light instead of accelerating into the intersection from a turn lane.

parameter count. Despite being optimized directly for the downstream task, this control achieves only a 2.86% ADE reduction. Together, these controls show that ADE improvement is not explained solely by adding adapter capacity on top of the scorer. Instead, the sparse SAE representation appears to provide a more useful interface for score correction, making relevant information more accessible than the dense query activation alone.

5 Conclusion

We applied mechanistic interpretability to E2E autonomous driving by training Sparse Autoencoders on activations of SMTS, a lightweight single-camera trajectory scoring planner. SAE latents are structured, visually grounded, and steerable, recovering route intent with 98.3% accuracy unsupervised, and an SAE-based scoring adapter reduces ADE@5s by 3.44%, confirming their practical utility.

Limitations – Our evaluation is entirely open-loop, so it does not measure how SAE features or interventions behave under closed-loop feedback. SMTS uses only a front-facing camera, leaving multi-camera or LiDAR-based planners unexplored. The counterfactual edit analysis depends on the quality of the object detector and image editing model, and imperfect edits may confound feature attribution. Finally, visual-edit responses and latent steering effects are positively aligned but moderate in magnitude, indicating that some visually responsive features do not translate cleanly into braking behavior.

Acknowledgments

This work used the Delta and DeltaAI systems at the National Center for Supercomputing Applications through allocation CIS260574 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants 2138259, 2138286, 2138307, 2137603, and 2138296 [40].

References

[1] World Health Organization. Global status report on road safety 2023. Technical report, World Health Organization, Geneva, Dec. 2023. URL https://www.who.int/publications/i/item/9789240086517. Accessed May 23, 2026.

[2] J. Slotkin. The data on self-driving cars is clear. we have to change course. *The New York Times*, Dec. 2025. URL https://www.nytimes.com/2025/12/02/opinion/self-driving-cars.html. Opinion.

[3] Wayve. LINGO-1: Exploring natural language for autonomous driving. https://wayve.ai/thinking/lingo-natural-language-autonomous-driving/, Sept. 2023. Wayve blog. Accessed May 23, 2026.

[4] A.-M. Marcu, L. Chen, J. Hünermann, A. Karnsund, B. Hanotte, P. Chidananda, S. Nair, V. Badrinarayanan, A. Kendall, J. Shotton, E. Arani, and O. Sinavski. Lingoqa: Visual question answering for autonomous driving, 2024. URL https://arxiv.org/abs/2312.14115.

[5] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li. End-to-End Autonomous Driving: Challenges and Frontiers . *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 46 (12):10164–10183, Dec. 2024. ISSN 1939-3539. doi:10.1109/TPAMI.2024.3435937. URL https://doi.ieeecomputersociety.org/10.1109/TPAMI.2024.3435937.

[6] R. Paleja, Y. Niu, A. Silva, C. Ritchie, S. Choi, and M. Gombolay. Learning interpretable, high-performing policies for autonomous driving, 2023. URL https://arxiv.org/abs/2202.02352.

[7] K. Renz, L. Chen, E. Arani, and O. Sinavski. Simlingo: Vision-only closed-loop autonomous driving with language-action alignment, 2025. URL https://arxiv.org/abs/2503.09594.

[8] C. Sima, K. Renz, K. Chitta, L. Chen, H. Zhang, C. Xie, J. Beißwenger, P. Luo, A. Geiger, and H. Li. Drivelm: Driving with graph visual question answering, 2025. URL https://arxiv.org/abs/2312.14150.

[9] K. Ding, B. Chen, Y. Su, H. ang Gao, B. Jin, C. Sima, W. Zhang, X. Li, P. Barsch, H. Li, and H. Zhao. Hint-ad: Holistically aligned interpretability in end-to-end autonomous driving, 2024. URL https://arxiv.org/abs/2409.06702.

[10] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li. Planning-oriented autonomous driving, 2023. URL https://arxiv.org/abs/2212.10156.

[11] X. Weng, B. Ivanovic, Y. Wang, Y. Wang, and M. Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15449–15458, 2024. doi:10.1109/CVPR52733.2024.01463.

[12] L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders, 2024. URL https://arxiv.org/abs/2406.04093.

[13] M. Lan, P. Torr, A. Meek, A. Khakzar, D. Krueger, and F. Barez. Quantifying feature space universality across large language models via sparse autoencoders, 2025. URL https://arxiv.org/abs/2410.06981.

[14] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL https://arxiv.org/abs/2309.08600.

[15] A. Swann, L. McGranahan, H. Buurmeijer, M. K. III, and M. Schwager. Sparse autoencoders reveal interpretable and steerable features in vla models, 2026. URL https://arxiv.org/abs/2603.19183.

[16] X. Jin, A. Chatterjee, P. Kumar, and R. Paleja. Event-grounded sparse autoencoders for vision-language-action policies, 2026. URL https://arxiv.org/abs/2605.17204.

[17] R. Xu, H. Lin, W. Jeon, H. Feng, Y. Zou, L. Sun, J. Gorman, E. Tolstaya, S. Tang, B. White, B. Sapp, M. Tan, J.-J. Hwang, and D. Anguelov. Wod-e2e: Waymo open dataset for end-to-end driving in challenging long-tail scenarios, 2025. URL https://arxiv.org/abs/2510.26125.

[18] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba. End to end learning for self-driving cars, 2016. URL https://arxiv.org/abs/1604.07316.

[19] B. Jiang, S. Chen, H. Gao, B. Liao, Q. Zhang, W. Liu, and X. Wang. Vadv2: End-to-end vectorized autonomous driving via probabilistic planning, 2026. URL https://arxiv.org/abs/2402.13243.

[20] Z. Li, W. Yao, Z. Wang, X. Sun, J. Chen, N. Chang, M. Shen, J. Song, Z. Wu, S. Lan, and J. M. Alvarez. Ztrs: Zero-imitation end-to-end autonomous driving with trajectory scoring, 2025. URL https://arxiv.org/abs/2510.24108.

[21] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving, 2022. URL https://arxiv.org/abs/2205.15997.

[22] B. Jaeger, K. Chitta, and A. Geiger. Hidden biases of end-to-end driving models, 2023. URL https://arxiv.org/abs/2306.07957.

[23] L. Nguyen, M. Fauth, B. Jaeger, D. Dauner, M. Igl, A. Geiger, and K. Chitta. Lead: Minimizing learner-expert asymmetry in end-to-end driving, 2026. URL https://arxiv.org/abs/2512.20563.

[24] Z. Xing, X. Zhang, Y. Hu, B. Jiang, T. He, Q. Zhang, X. Long, and W. Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving, 2025. URL https://arxiv.org/abs/2503.05689.

[25] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, Y.-G. Jiang, and J. M. Alvarez. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation, 2024. URL https://arxiv.org/abs/2406.06978.

[26] K. Li, Z. Li, S. Lan, Y. Xie, Z. Zhang, J. Liu, Z. Wu, Z. Yu, and J. M. Alvarez. Hydra-mdp++: Advancing end-to-end driving via expert-guided hydra-distillation, 2025. URL https://arxiv.org/abs/2503.12820.

[27] W. Yao, Z. Li, S. Lan, Z. Wang, X. Sun, J. M. Alvarez, and Z. Wu. Drivesuprim: Towards precise trajectory selection for end-to-end planning, 2025. URL https://arxiv.org/abs/2506.06659.

[28] E. Kirby, A. Boulch, Y. Xu, Y. Yin, G. Puy, Éloi Zablocki, A. Bursuc, S. Gidaris, R. Marlet, F. Bartoccioni, A.-Q. Cao, N. Samet, T.-H. VU, and M. Cord. Driving on registers, 2026. URL https://arxiv.org/abs/2601.05083.

[29] Z. Li, W. Yao, Z. Wang, X. Sun, J. Chen, N. Chang, M. Shen, Z. Wu, S. Lan, and J. M. Alvarez. Generalized trajectory scoring for end-to-end multimodal planning, 2025. URL https://arxiv.org/abs/2506.06664.

[30] K. Guo, H. Liu, X. Wu, J. Pan, and C. Lv. ipad: Iterative proposal-centric end-to-end autonomous driving, 2025. URL https://arxiv.org/abs/2505.15111.

[31] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding, L. Feng, G. Heinrich, J. Huang, P. Karkus, B. Li, P. Li, T.-Y. Lin, D. Liu, M.-Y. Liu, L. Liu, Z. Liu, J. Lu, Y. Mao, P. Molchanov, L. Pavao, Z. Peng, M. Ranzinger, E. Schmerling, S. Shen, Y. Shi, S. Tariq, R. Tian, T. Wekel, X. Weng, T. Xiao, E. Yang, X. Yang, Y. You, X. Zeng, W. Zhang, B. Ivanovic, and M. Pavone. Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail, 2026. URL https://arxiv.org/abs/2511.00088.

[32] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

[33] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, R. Grosse, S. McCandlish, J. Kaplan, D. Amodei, M. Wattenberg, and C. Olah. Toy models of superposition, 2022. URL https://arxiv.org/abs/2209.10652.

[34] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models, 2025. URL https://arxiv.org/abs/2403.19647.

[35] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving?, 2024. URL https://arxiv.org/abs/2312.03031.

[36] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang. Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscenes. *arXiv preprint arXiv:2305.10430*, 2023.

[37] D. Bolya, P.-Y. Huang, P. Sun, J. H. Cho, A. Madotto, C. Wei, T. Ma, J. Zhi, J. Rajasegaran, H. Rasheed, J. Wang, M. Monteiro, H. Xu, S. Dong, N. Ravi, D. Li, P. Dollár, and C. Feichtenhofer. Perception encoder: The best visual embeddings are not at the output of the network. *arXiv*, 2025.

[38] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuscenes: A multimodal dataset for autonomous driving. *CoRR*, abs/1903.11027, 2019. URL http://arxiv.org/abs/1903.11027.

[39] A. Clauset, M. E. J. Newman, and C. Moore. Finding community structure in very large networks. *Physical Review E*, 70(6):066111, 2004. doi:10.1103/PhysRevE.70.066111.

[40] T. J. Boerner, S. Deems, T. R. Furlani, S. L. Knuth, and J. Towns. ACCESS: Advancing innovation: NSF's advanced cyberinfrastructure coordination ecosystem: Services & support. In *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good*, 2023. doi:10.1145/3569951.3597559.

[41] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao. Depth anything v2, 2024. URL https://arxiv.org/abs/2406.09414.

[42] M. E. J. Newman and M. Girvan. Finding and evaluating community structure in networks. *Physical Review E*, 69(2):026113, 2004. doi:10.1103/PhysRevE.69.026113.

[43] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee. Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection, 2026. URL https://arxiv.org/abs/2509.25164.

[44] Q. Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

[45] S. I. Team, C. Qiao, C. Hui, C. Li, C. Wang, D. Song, J. Zhang, J. Li, Q. Xiang, R. Wang, S. Sun, W. Zhu, X. Tang, Y. Hu, Y. Chen, Y. Huang, Y. Duan, Z. Chen, and Z. Guo. Fireredimage-edit-1.0 technical report, 2026. URL https://arxiv.org/abs/2602.13344.

A Implementation Details

Dataset and input representation. We use version 1.0.0 of the Waymo Open Dataset End-to-End Driving (WOD-E2E) camera dataset [17]. The complete indexed training and validation splits contain 415,663 and 106,360 samples, respectively. We train our end-to-end planning model (labeled SMTS) on 250,000 samples from the training split. The SAEs are trained using activations extracted from the complete training split.

SMTS uses only the front-facing camera image. We decode each JPEG image to an RGB tensor, convert its values from uint8 to $[0, 1]$, and apply the evaluation preprocessing associated with the pretrained `timm/vit_pe_spatial_small_patch16_512.fb` backbone [37]. This consists of bicubic resizing and center cropping to 512×512, followed by channel-wise normalization with $\boldsymbol{\mu} = (0.5, 0.5, 0.5)$ and $\boldsymbol{\sigma} = (0.5, 0.5, 0.5)$. We do not apply deterministic or stochastic image augmentation.

Each sample contains a past ego state tensor $\mathbf{p} \in \mathbb{R}^{16\times 6}$, where the six state variables are $(x, y, v_x, v_y, a_x, a_y)$. The 16 states are sampled at 4 Hz and represent four seconds of history. Route intent is represented as a three-dimensional one-hot vector corresponding to {STRAIGHT, LEFT, RIGHT}. The flattened 96-dimensional state history and three-dimensional intent vector are concatenated to form the planner conditioning input. The model predicts $T = 20$ future positions at 4 Hz, corresponding to a five-second prediction horizon, and generates $K = 50$ trajectory proposals.

SMTS architecture and training. We use the pretrained `timm/vit_pe_spatial_small_patch16_512.fb` vision transformer as the visual backbone [37]. The backbone is frozen throughout training, while the visual adapter and planner are trainable. We use the backbone's final spatial feature map, which contains 32×32 visual tokens with feature dimension $d = 384$.

The planner contains four transformer blocks. Each block updates one planner-query token using eight-head self-attention, eight-head cross-attention over the visual tokens, and an MLP with hidden dimension $4d$. The final query token is decoded into $K = 50$ sequences of raw acceleration and angular-velocity controls. For proposal k and timestep t, the raw controls $(\tilde{a}_{kt}, \tilde{\omega}_{kt})$ are bounded as

$$a_{kt} = a_{\max} \tanh(\tilde{a}_{kt}), \qquad \omega_{kt} = \omega_{\max} \tanh(\tilde{\omega}_{kt}), \tag{2}$$

where $a_{\max} = 8.0\,\mathrm{m/s^2}$ and $\omega_{\max} = 1.0\,\mathrm{rad/s}$. The initial position, speed, and heading are obtained from the final observed ego state:

$$x_{k0} = x_{\text{past}}, \quad y_{k0} = y_{\text{past}}, \quad v_{k0} = \sqrt{v_{x,\text{past}}^2 + v_{y,\text{past}}^2 + \varepsilon_v}, \quad \theta_{k0} = \operatorname{atan2}(v_{y,\text{past}}, v_{x,\text{past}}), \tag{3}$$

with $\varepsilon_v = 10^{-6}$. We then apply the differentiable kinematic rollout

$$x_{k,t+1} = x_{kt} + v_{kt}\cos(\theta_{kt})\Delta t, \tag{4}$$
$$y_{k,t+1} = y_{kt} + v_{kt}\sin(\theta_{kt})\Delta t, \tag{5}$$
$$\theta_{k,t+1} = \theta_{kt} + \omega_{kt}\Delta t, \tag{6}$$
$$v_{k,t+1} = \max(v_{kt} + a_{kt}\Delta t,\, 0)\,, \tag{7}$$

using $\Delta t = 0.25\,\mathrm{s}$ for $T = 20$ steps. Thus, each proposal represents a 5-second trajectory. More precisely, the implementation directly predicts yaw rate rather than steering angle and therefore does not explicitly model wheelbase.

For proposal k of sample i, we define its average displacement error as

$$e_{ik} = \frac{1}{T}\sum_{t=1}^{T} \left\|\hat{\mathbf{y}}_{ikt} - \mathbf{y}_{it}\right\|_2 . \tag{8}$$

The trajectory loss is a top-5 winner-take-all objective,

$$\mathcal{L}_{\mathrm{traj}} = \frac{1}{5B}\sum_{i=1}^{B}\sum_{k\in\mathcal{I}_5(i)} e_{ik}, \tag{9}$$

where $\mathcal{I}_5(i)$ contains the five proposals with the lowest ADE. The proposal scorer predicts the ADE of each proposal and is trained using detached regression targets:

$$\mathcal{L}_{\mathrm{score}} = \frac{1}{BK}\sum_{i=1}^{B}\sum_{k=1}^{K} \left(s_{ik} - \mathrm{detach}(e_{ik})\right)^2 . \tag{10}$$

Detaching the ADE targets and proposal trajectories prevents the scoring loss from modifying the trajectory generator.

Adversarial scorer supervision. To improve scorer robustness, we generate adversarial trajectories directly in Cartesian trajectory space during training. Starting from the detached proposal $\tilde{\mathbf{y}}_{ik}^{(0)} = \mathrm{detach}(\hat{\mathbf{y}}_{ik})$, we perform three projected gradient-ascent steps:

$$\tilde{\mathbf{y}}_{ik}^{(m+1)} = \Pi_{\mathcal{B}_\infty(\hat{\mathbf{y}}_{ik},\epsilon)}\left[\tilde{\mathbf{y}}_{ik}^{(m)} + \alpha\ \mathrm{sign}\Big(\nabla_{\tilde{\mathbf{y}}_{ik}^{(m)}} s_{ik}^{(m)}\Big)\right], \tag{11}$$

where $\epsilon = 0.1$, $\alpha = \epsilon/3$, and $\Pi_{\mathcal{B}_\infty}$ clips every trajectory coordinate to remain within ϵ of the original proposal. Gradient ascent increases the scorer's predicted ADE, producing locally difficult trajectories that improve the generalization of the scoring module to trajectories not present in the original training dataset. We recompute the true ADE $\tilde{e}_{ik}$ of each perturbed trajectory and train the scorer to predict it:

$$\mathcal{L}_{\mathrm{adv}} = \frac{1}{BK}\sum_{i=1}^{B}\sum_{k=1}^{K} \left(\tilde{s}_{ik} - \mathrm{detach}(\tilde{e}_{ik})\right)^2 . \tag{12}$$

Depth prediction head. The model additionally contains an auxiliary depth head operating on the adapted visual feature map. The head uses two convolutional layers with GELU activations and nearest-neighbor upsampling, followed by a convolution and softplus activation, producing a 128×128 depth map. A frozen `Depth-Anything-V2-Small` model [41] generates pseudo-depth targets from the front image, and the auxiliary objective is

$$\mathcal{L}_{\mathrm{depth}} = \frac{1}{BHW}\sum_{i=1}^{B} \left\|\hat{\mathbf{D}}_i - \mathrm{detach}(\mathbf{D}_i^*)\right\|_1 . \tag{13}$$

We train SMTS for 10 epochs using Adam with learning rate 10^{-4}, no learning rate scheduling, and a global batch size of 128 on 4× NVIDIA A100 GPUs. The reported checkpoint uses

$$\mathcal{L}_{\mathrm{SMTS}} = \mathcal{L}_{\mathrm{traj}} + \mathcal{L}_{\mathrm{score}} + 0.1\,\mathcal{L}_{\mathrm{adv}} + 0.1\,\mathcal{L}_{\mathrm{depth}} \tag{14}$$

Sparse autoencoders. We independently train one sparse autoencoder (SAE) on the post-transformer query activations from each SMTS block $\ell \in \{0, 1, 2, 3\}$. Each frame contributes one 384-dimensional activation token per block. Consequently, each SAE is trained using 415,663 training tokens and evaluated using 106,360 validation tokens.

Each SAE has a latent dimension of 384. The encoder uses Top-k sparsification with $k = 24$ and no encoder bias. The AuxK objective uses $k_{\text{aux}} = 512$, restricted to currently dead latents, with coefficient $\alpha_{\text{aux}} = 1/32$. A latent is considered dead when it has not activated for 500 consecutive optimizer steps.

Before encoding, we subtract a learned preprocessing bias initialized using an approximate geometric median of 10,000 training tokens. For each activation, we then subtract its mean across feature dimensions and divide by its centered L_2 norm. Reconstruction loss is normalized by the centered mean-squared activation magnitude. Decoder columns are renormalized to unit norm after every optimizer update.

The SAE objective is

$$\mathcal{L}_{\text{SAE}} = \frac{\text{MSE}(\hat{\mathbf{x}}, \mathbf{x})}{\text{CMSE}(\mathbf{x})} + \frac{1}{32} \frac{\text{MSE}(\hat{\mathbf{r}}_{\text{aux}}, \mathbf{r})}{\text{CMSE}(\mathbf{x})}, \tag{15}$$

where $\mathbf{r}$ is the reconstruction residual and $\hat{\mathbf{r}}_{\text{aux}}$ is reconstructed using dead latents.

We train each SAE for 100 epochs using Adam with learning rate 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.999)$, batch size 4096, gradient clipping at norm 1.0, and random seed 42. We retain the checkpoint with the lowest validation loss.

B Open-Loop Planning Performance

We compare against Latent TransFuser [21], GTRS-Dense [29], and DrivoR [28]. These models were developed for other datasets, so re-train DrivoR and GTRS-Dense on the same 250,000-sample subset. These models were chosen for comparison to SMTS as they are closest to SMTS in design and parameter count.

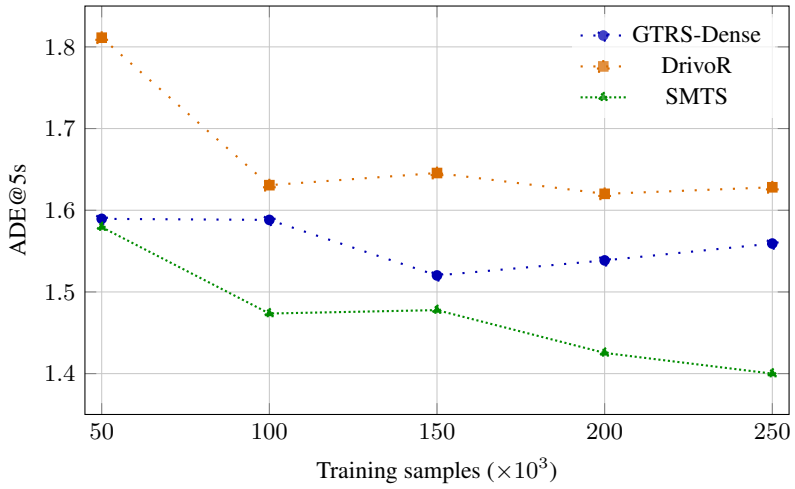


Figure 6: **Data scaling.** Selected trajectory ADE@5s on the WOD-E2E validation split as a function of training set size. SMTS shows more consistent gains from additional training data than the compared trajectory scoring baselines and achieves the lowest error overall.

Figure 6 shows that SMTS benefits more consistently from additional training data than the compared baselines. While SMTS performs similarly to GTRS-Dense at 50,000 training samples, its ADE decreases more consistently as the dataset size increases, ultimately achieving the lowest ADE at 250,000 samples. Furthermore, SMTS obtains the highest average RFS and leads on 6 of 11

Scene subset	GTRS-Dense	DrivoR	SMTS (ours)
Average	7.688	7.645	**7.765**
Construction	8.080	8.147	**8.343**
Intersection	**7.948**	7.632	7.706
Pedestrian	7.782	7.777	**7.977**
Cyclist	7.613	**7.636**	7.569
Multi-lane maneuver	7.409	7.483	**7.614**
Single-lane maneuver	8.194	8.037	**8.299**
Cut-in	7.821	**7.867**	7.715
Foreign object debris	7.891	7.860	**7.988**
Special vehicle	8.030	7.687	**8.078**
Spotlight	6.295	**6.773**	6.749
Others	**7.499**	7.193	7.372

Table 2: **WOD-E2E test-set Rater Feedback Score by scene subset** (higher is better). All models are re-trained on the same 250,000-sample training subset and scored by the private WOD-E2E test set evaluator.

scene subsets (Table 2), with the largest margins on construction, pedestrian, and multi-lane maneuver scenes.

C Feature Clustering Details

We analyze co-activation structure among the block-3 SAE features on all $N = 106{,}360$ WOD-E2E validation scenes. We use block 3 because it is the final planner representation immediately preceding trajectory generation and proposal scoring.

C.1 Activation Filtering

Let $z_{ij} \geq 0$ denote the activation of SAE feature j on validation scene i. We define the binary firing indicator and activation frequency as

$$b_{ij} = \mathbb{1}[z_{ij} > 0], \qquad f_j = \frac{1}{N} \sum_{i=1}^{N} b_{ij}. \tag{16}$$

We retain features satisfying

$$0.005 \leq f_j \leq 0.50. \tag{17}$$

The lower bound removes features that fire too rarely to support reliable correlation estimates, while the upper bound removes features that fire for many scenes and do not provide information specific to a scene.

Of the 384 block 3 SAE features, 258 are retained after filtering, as 125 features with activation frequencies below 0.5% and one feature with an activation frequency above 50% are excluded. Among the retained features, the observed activation frequencies range from 0.509% to 46.542%.

For every retained feature pair (j, ℓ), we compute the Pearson correlation between their binary firing indicators:

$$\rho_{j\ell} = \operatorname{corr}\left(\mathbf{b}_{:j}, \mathbf{b}_{:\ell}\right). \tag{18}$$

For binary variables, this correlation is equivalent to the phi coefficient and measures whether two features fire on the same scenes more frequently than expected from their marginal firing rates.

We construct an undirected graph in which retained features are nodes and an edge connects j and ℓ when

$$\rho_{j\ell} \geq \tau. \tag{19}$$

Clusters are the graph's connected components, and components containing fewer than three features are discarded. Because connected components use transitive connectivity, not every feature pair within a cluster is required to have correlation above τ.

Table 3: Sensitivity of block 3 SAE feature clusters to the co-activation correlation threshold. Cluster sizes are reported in descending order. Intent purity is measured among each cluster's top 50 validation scenes by activation.

Threshold τ	Number of clusters	Cluster sizes	Clustered features	Top 50 intent purity
0.20	1	238	238	100%, collapsed component
0.35	4	64, 36, 34, 27	161	100% for all clusters
0.40	6	43, 29, 27, 23, 3, 3	128	96%–100%

Table 4: Block-3 SAE cluster summaries at the primary threshold $\tau = 0.35$. Intent and standardized scene statistics are computed over each cluster's top 50 validation scenes by activation.

Cluster	Features	Top-50 intent	Interpretation	Speed z	Braking z	Score entropy z
C0	64	Straight (100%)	Slow straight driving	-1.04	-0.52	$+0.03$
C1	36	Left (100%)	Slow left turn	-1.07	$+2.00$	-1.75
C2	34	Straight (100%)	High-speed straight driving	$+0.98$	-0.30	$+0.48$
C3	27	Right (100%)	Slow right turn	-1.04	$+1.12$	-0.94

C.2 Robustness of the Clustering Procedure

We evaluate correlation thresholds $\tau \in \{0.20, 0.35, 0.40\}$ and use $\tau = 0.35$ for the primary analysis. Table 3 summarizes the resulting cluster structures.

At $\tau = 0.20$, weak correlation paths transitively connect 238 of the 258 retained features into one large component. Although its highest activating scenes all have straight intent, this component combines otherwise distinct driving modes and is therefore too coarse for interpretation.

The four primary clusters identified at $\tau = 0.35$ remain present at $\tau = 0.40$. The stopped/slow-straight, stopped left turn, high speed straight, and stopped right turn clusters retain respectively $43/64$, $27/36$, $29/34$, and $23/27$ of their features. Their dominant intents remain unchanged, and each retains 100% top-50 intent purity. Intent purity is the fraction of a cluster's top 50 validation scenes, by activation, that share the most common intent label. For example, a purity of 100% with dominant intent "Left" means that all 50 scenes are labeled LEFT. The two additional components at $\tau = 0.40$ contain only three features each. Thus, the principal intent-aligned structure is stable across the two stricter thresholds, while $\tau = 0.35$ preserves more features within each semantic mode.

As an additional robustness check, we cluster the retained features using greedy modularity maximization, which does not require a correlation threshold [39, 42]. After matching each recovered community to the connected-component cluster with which it has the greatest overlap, 94.3% of retained features receive the same label under both procedures. The dominant cluster structure is therefore largely preserved under an alternative clustering method.

C.3 Cluster-Level Scene Summaries

For a cluster $\mathcal{C}$, we define its activation score on scene i using the sum of its raw SAE activation magnitudes:

$$A_i(\mathcal{C}) = \sum_{j \in \mathcal{C}} z_{ij}. \tag{20}$$

We rank validation scenes by $A_i(\mathcal{C})$ and summarize each cluster using its top 50 scenes. This ranking uses activation magnitude only after clusters have been constructed from binary co-activation correlations.

Table 4 reports the four primary clusters at $\tau = 0.35$. Continuous scene statistics are standardized relative to their distribution over the complete validation split.

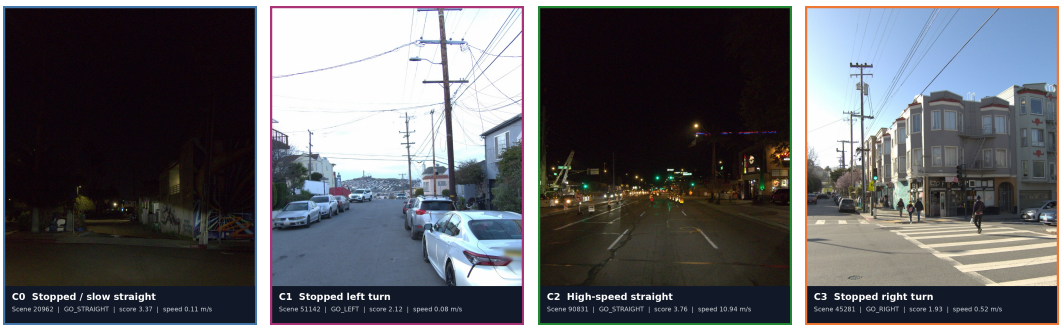


Figure 7: Highest activating validation scene for each feature cluster. The clusters capture distinct combinations of route intent and vehicle motion.

The clusters separate both route intent and dynamic driving state. C1 and C3 distinguish left- and right-turn intent despite both primarily activating in slow or stationary scenes. C0 and C2 share straight intent but separate stationary and high-speed motion. These results show that the final planner representation is organized into distributed feature groups encoding combinations of route intent and ego motion.

D Input Conditioning Analysis

We test whether SAE latents depend on visual input beyond the route intent and ego history used to initialize the query. We sample 9,000 WOD-E2E validation frames, balanced across the six combinations of original route intent $\{\text{STRAIGHT}, \text{LEFT}, \text{RIGHT}\}$ and whether the vehicle is moving or stopped, with 1,500 frames from each group. This balancing prevents common driving conditions from dominating the analysis.

For each sampled image, we keep the image unchanged and rerun the planner with each of the three route intents and two standardized ego histories. The first history represents a stopped vehicle, with position, velocity, and acceleration set to zero. The second represents steady forward motion at 5.96 m/s, with zero acceleration and positions advanced at 0.25 s intervals. The cruise speed of 5.96 m/s was chosen as the median speed over all moving (where moving is defined as having a speed greater than 0.5 m/s) validation frames. Each image is therefore evaluated under the same six combinations of intent and ego history, producing $54{,}000$ latent vectors.

We then use a three-way ANOVA to separate variation in the latent representation associated with image, route intent, ego history, and their interactions. For each latent dimension, we compute the sum of squares associated with each term, sum these values across dimensions, and normalize by the total sum of squares. The resulting percentages are reported in Table 1.

The main effect of image measures how much the latent representation differs across images after averaging over intent and ego history. The image$\times$intent term measures how much the effect of route intent differs from image to image, with the remaining interaction terms interpreted similarly. Because every image is evaluated under every intent and ego history combination, these effects are not caused by differences in the metadata paired with each image.

E Counterfactual Visual Edit Details

This section details the counterfactual image-editing pipeline we use to examine the visual responsiveness of SAE features for our end-to-end driving model.

E.1 Edit generation pipeline

For each scene we operate on the front camera image and apply a four-stage pipeline:

1. **Object detection.** We run a YOLO detector (`yolo26x`) on the front image to obtain a structured scene description (object classes, confidences, and bounding boxes) [43].
2. **Edit proposal (VLM).** The scene description and image are passed to a Qwen3.5-4B vision–language model (VLM) [44]. The VLM is constrained by a fixed system prompt to propose at most one counterfactual edit and to return JSON with the schema (`edit_type`, `edit_direction`, `prompt`).
3. **Allowed edits.** Proposals are validated against a closed vocabulary. The permitted edit types are traffic light color, traffic light presence, pedestrian presence, and stop sign presence or location; the permitted directions are `green_to_red`, `red_to_green`, `yellow_to_green`, `add_traffic_light`, `remove_traffic_light`, `add_pedestrian`, `remove_pedestrian`, `add_stop_sign`, `remove_stop_sign`, and `move_stop_sign`. Any scene with no applicable edit, or with an unparseable/invalid proposal, is skipped.
4. **Image editing.** Valid prompts are rendered by an instruction-guided diffusion image editor (`FireRed-Image-Edit`) [45]. We manually engineered the image generation system prompt to ensure instruction following and minimize unrelated changes to the image.

Replay protocol. To measure the effect of an edit, we substitute the edited image for the original front camera frame, keep all other inputs (past state and intent) fixed, and rerun the planner. We extract the block 3 query token for both the original and edited inputs, encode both with the SAE, and record the per-feature activation change $\Delta z_i = z_i^{\text{edited}} - z_i^{\text{original}}$, which we then normalize across the validation split. This isolates how a targeted visual change propagates into the SAE feature space and, in turn, the planner output.

E.2 Edit category counts

We processed 1000 validation scenes. The VLM proposed an applicable edit for 615 of them and returned `no_change` for the remaining 385 (21 of which were rejected for invalid edit specifications).

Edit direction	Count
green → red	193
red → green	95
add stop sign	46
add pedestrian	145
no change / rejected	385

Table 5: Counterfactual visual edit dataset summary. The four directions above are those used in the main edit sensitivity analysis. A further 136 edits in other categories were also generated (`add_traffic_light` 46, `yellow_to_green` 41, `remove_stop_sign` 25, `remove_pedestrian` 14, `remove_traffic_light` 7, `move_stop_sign` 3), resulting in 615 edited images in total.

E.3 Additional examples

Figure 8 shows representative successful edits as original/edited image pairs. For each pair we overlay the planner's predicted trajectory before and after the edit and list the top SAE feature activation changes Δz_i. Stop-inducing edits (e.g. green→red, add stop sign, add pedestrian) shorten or add braking to the predicted trajectory and increase the corresponding stop-associated SAE features.

E.4 Failure cases

For transparency, we also show failure modes of our pipeline (Figure 9): (i) *unrealistic edits*, where the editor introduces visually implausible content; (ii) *irrelevant-object edits*, where the change is

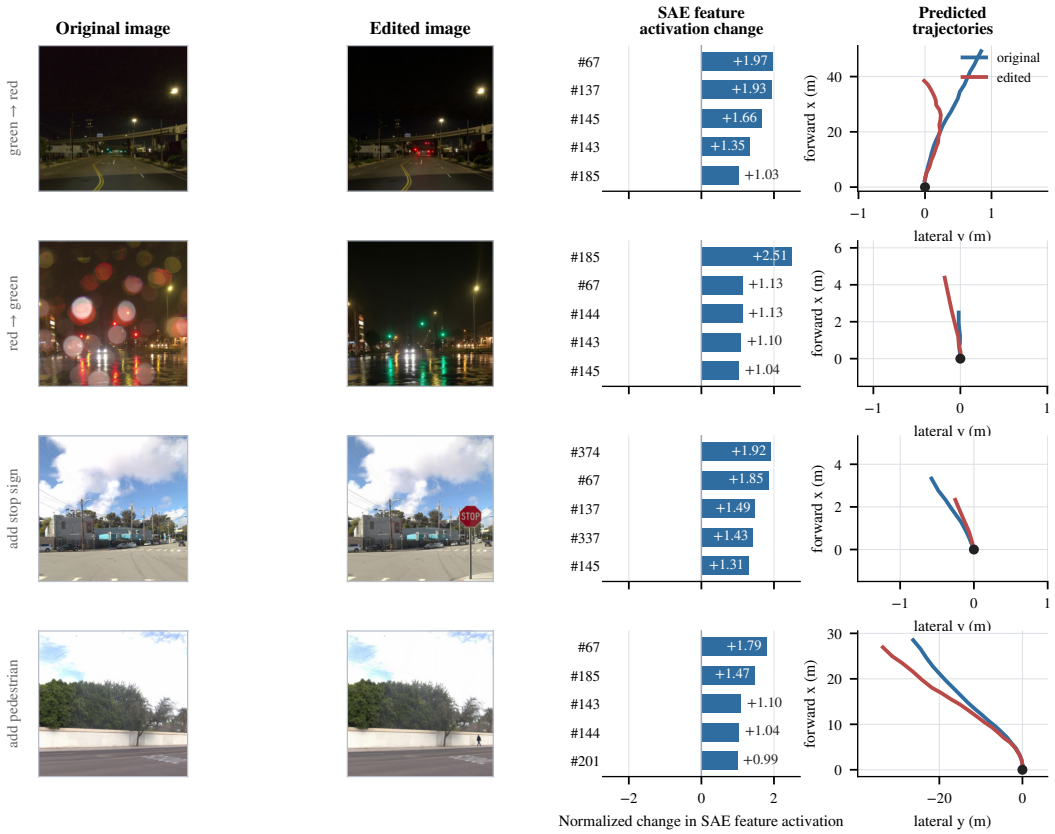


Figure 8: Additional counterfactual visual edit examples. Each row shows the original front camera image, the edited image, the top 5 SAE feature activation changes for that edit direction (normalized Δz_i), and the planner's predicted trajectory before (blue) and after (red) the edit. Stop-inducing edits shorten or brake the predicted path; `red`→`green` extends it.

applied to an object that does not affect driving; and (iii) *behaviorally inert edits*, where a visible image change does not alter the predicted trajectory. These cases motivate our focus on stop-inducing edits, for which the visual-to-behavior link is strongest and most consistent.

F Latent Steering

F.1 Metrics

Define $p_{k,\tau} \in \mathbb{R}^2$ to be proposal k at horizon step τ, s_k its score, and $\hat{k} = \arg\min_k s_k$ the selected proposal. For the selected trajectory, we define $\Delta p_\tau = p_{\hat{k},\tau} - p_{\hat{k},\tau-1}$, $\phi_\tau = \operatorname{atan2}(\Delta p_{\tau,y}, \Delta p_{\tau,x})$, and let a_τ denote longitudinal control.

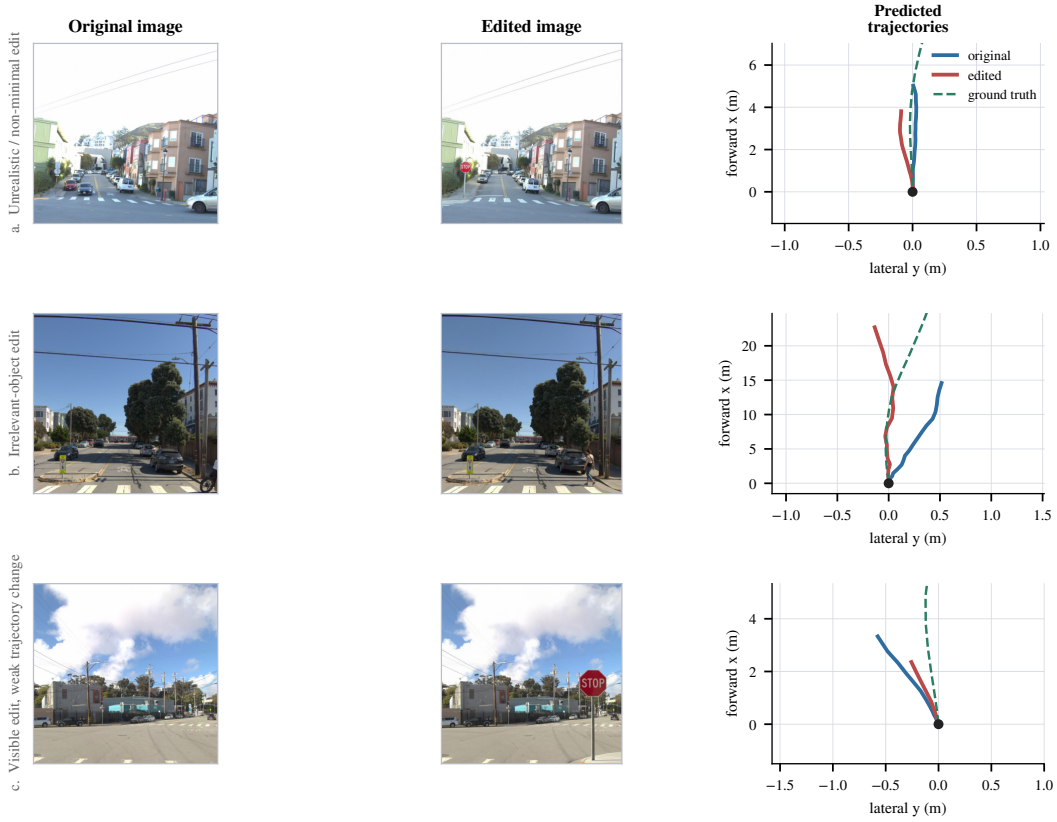


Figure 9: Representative failure modes of the counterfactual editing pipeline. **(a)** Unrealistic edit: an oddly placed stop sign is added, and the editor warps the scene and deletes parked cars. **(b)** Irrelevant object edit: a pedestrian is added on the right sidewalk, away from the intended driving path. **(c)** Visible edit, weak trajectory change: a clear stop sign is added, yet the predicted path shortens only slightly relative to the original. Note that blue/red curves are the selected planner proposals before/after the edit and the dashed green curve is the ground truth future trajectory.

Metric	Definition
Final lateral displacement	$d_{\text{lat}} = p_{\hat{k},T,y}$
Average curvature	$\bar{\kappa} = \dfrac{1}{T-2}\displaystyle\sum_{\tau=2}^{T-1} \dfrac{\lvert\phi_\tau - \phi_{\tau-1}\rvert}{\frac{1}{2}(\lVert\Delta p_\tau\rVert + \lVert\Delta p_{\tau-1}\rVert) + \epsilon}$
Brake magnitude	$\beta = \dfrac{1}{T}\displaystyle\sum_{\tau=1}^{T} \max(0, -a_\tau)$
Acceleration magnitude	$\gamma = \dfrac{1}{T}\displaystyle\sum_{\tau=1}^{T} \max(0, a_\tau)$
Score margin	$m = s_{(2)} - s_{(1)}$
Proposal spread	$\sigma_{\text{prop}} = \dfrac{1}{KT}\displaystyle\sum_{k=1}^{K}\sum_{\tau=1}^{T} \lVert p_{k,\tau} - \bar{p}_\tau\rVert_2$

Table 6: Metrics recorded at each intervention level.

F.2 Feature Quality

The quality of an SAE feature as a controller is calculated through the following measures:

Standardized Response Magnitude. For each SAE feature i and target statistic m, we measure the change induced by the maximum intervention level, averaged over the scenes where feature i is most active. Let S_i denote this set of scenes, and let $y_m^{(s)}(\alpha)$ be the value of statistic m for scene s after intervening on feature i with strength α. We define the response per scene at the maximum intervention level as

$$\Delta_{i,m}^{(s)} = y_m^{(s)}(\alpha_{\max}) - y_m^{(s)}(0).$$

To make effects comparable across statistics with different units, we normalize by the standard deviation across the validation split, σ_m, of statistic m:

$$E_{i,m} = \frac{\frac{1}{|S_i|}\sum_{s\in S_i}\Delta_{i,m}^{(s)}}{\sigma_m}.$$

The absolute standardized response magnitude is then

$$A_{i,m} = |E_{i,m}|.$$

Monotonicity. A feature intervention is more interpretable if increasing the intervention strength produces an ordered change in the target statistic. For each scene s, we compute the Spearman rank correlation between the intervention levels and the resulting statistic values:

$$\rho_{i,m}^{(s)} = \text{Spearman}\left([\alpha_0,\ldots,\alpha_L],[y_m^{(s)}(\alpha_0),\ldots,y_m^{(s)}(\alpha_L)]\right).$$

The feature-level monotonicity score is the average Spearman correlation across scenes:

$$R_{i,m} = \frac{1}{|S_i|}\sum_{s\in S_i}\rho_{i,m}^{(s)}.$$

Since both increasing and decreasing monotonic responses indicate controllability, the final controllability score uses $|R_{i,m}|$.

Response Consistency. A large average response may be caused by a small number of outlier scenes. To penalize this, we define response consistency as the fraction of scenes whose response agrees with the average direction of effect. Let

$$g_{i,m} = \text{sign}(E_{i,m}).$$

A scene is counted as consistent if its response curve is monotonic in the global direction and if its response to the maximal intervention has the same sign as the global response:

$$g_{i,m}\rho_{i,m}^{(s)} > 0.5 \quad \text{and} \quad g_{i,m}\Delta_{i,m}^{(s)} > 0.$$

The response consistency score is

$$C_{i,m} = \frac{1}{|S_i|}\sum_{s\in S_i}\mathbf{1}\left[g_{i,m}\rho_{i,m}^{(s)} > 0.5 \ \wedge \ g_{i,m}\Delta_{i,m}^{(s)} > 0\right].$$

Response Selectivity. We also require a controllable feature to affect the target statistic more strongly than the other measured planner statistics. Let $\mathcal{M}$ be the set of target statistics:

$$\mathcal{M} = \{\text{final lateral displacement, average curvature, brake magnitude,}$$
$$\text{acceleration magnitude, score margin, proposal spread}\}$$

For each feature-statistic pair, we define response selectivity as the share of the feature's total standardized effect assigned to statistic m:

$$Q_{i,m} = \frac{A_{i,m}}{\sum_{m'\in\mathcal{M}}A_{i,m'} + \epsilon}.$$

This term is large when feature i affects statistic m more selectively than the other measured statistics.

Target statistic	Feature	Control score	Effect size	Monotonicity	Consistency	Selectivity
Brake magnitude	137	0.481	+0.715	0.941	1.000	0.715
Curvature	118	0.113	+0.472	0.509	0.641	0.736
Lateral displacement	326	0.330	+0.431	0.947	0.953	0.848
Score margin	228	1.440	+3.028	0.959	0.828	0.599
Proposal spread	250	0.410	+0.651	1.000	1.000	0.629

Table 7: Top robust block-3 SAE feature for each planner statistic. Effect sizes are normalized by the validation-set standard deviation of the corresponding statistic.

Controllability Score. Finally, we combine standardized response magnitude, monotonicity, response consistency, and response selectivity into a scalar controllability score:

$$\mathrm{Ctrl}_{i,m} = A_{i,m} \cdot |R_{i,m}| \cdot C_{i,m} \cdot Q_{i,m}.$$

Features are ranked separately for each statistic according to $\mathrm{Ctrl}_{i,m}$.

We perform an exhaustive intervention sweep to test whether latent-steering behavior generalizes beyond an individually selected feature or scene. Unless otherwise stated, we intervene on the block-3 SAE latents and evaluate the 64 validation scenes with the highest natural activation for each feature. Of the 384 SAE features, 340 activate on at least one validation scene.

F.3 Intervention Protocol and Single-Feature Responses

For feature j, we define its intervention scale as

$$s_j = \max\left(\mathrm{Std}[z_j \mid z_j > 0], \frac{1}{4}\,\mathrm{Mean}[z_j \mid z_j > 0], 0.05\right). \tag{21}$$

We increase the feature activation according to

$$z_j^{(\alpha)} = \max\left(0, z_j + \alpha s_j\right), \qquad \alpha \in \{0, 0.5, 1, 2\}, \tag{22}$$

while leaving all other SAE latents unchanged. The modified latent vector is decoded back into the reconstructed query and passed through the remaining planner heads. For each intervention, we measure the trajectory and scoring statistics defined in Section F.1.

F.4 Top Controllable Features

We rank features using the controllability score defined in Section F.2. Table 7 reports the strongest robust feature for each statistic, restricting the ranking to features with consistency of at least 0.5. In total, 254 of the 340 active block 3 features obtain a control score above 0.01, and 68 obtain a score above 0.1.

The next highest robust braking features are 87 and 114; the next highest lateral-displacement features are 207 and 265; and the next highest proposal spread features are 153 and 116. Score margin feature 228 is the strongest robust scorer-related feature. Features 137, 228, and 250 belong to clusters C2, C0, and C1, respectively, while features 118 and 326 are unclustered at the correlation threshold used in Section C.3.

The metrics also split into relating to the generation of trajectories and/or scoring of trajectories. Lateral displacement, curvature, braking, and proposal spread characterize generated controls or trajectories, whereas score margin characterizes the proposal scorer. Because the intervention is applied to the shared planner query, this comparison measures differential output responses rather than strictly head-isolated causal effects.

F.5 Cluster-Level Steering Effects

We average each control score over the features assigned to each co-activation cluster. The mean control scores are 0.0794, 0.0799, 0.0647, and 0.0671 for the best feature in each cluster C0–C3,

respectively. C1 has the strongest mean proposal spread response (0.054), while C0 exhibits the strongest clustered score margin response (0.031).

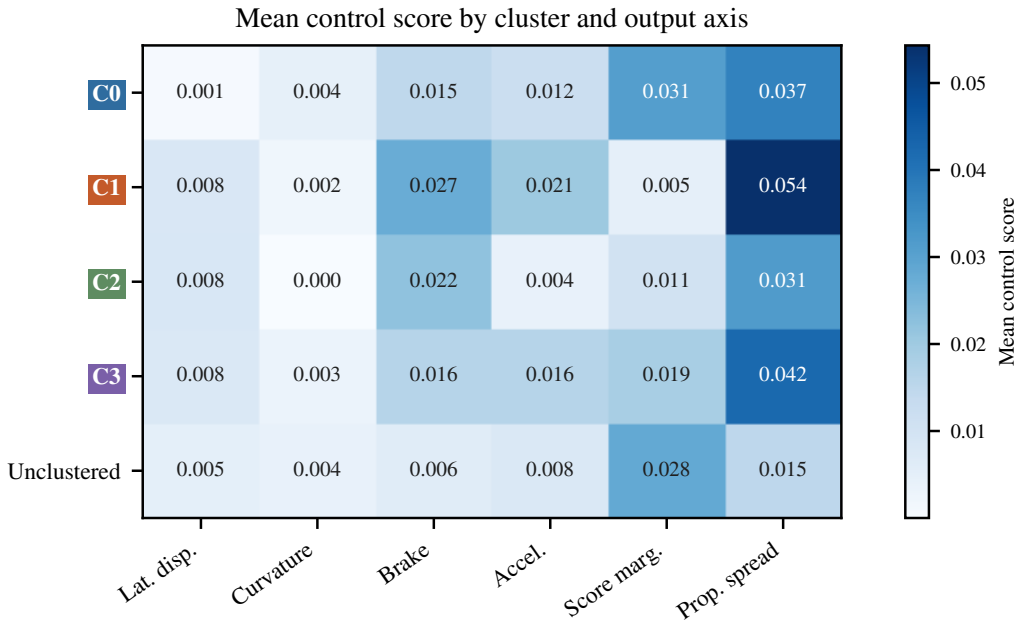


Figure 10: Mean control score for each block 3 feature cluster and planner statistic. Clusters differ in their secondary controlled statistics.

F.6 Natural Association Versus Steering Response

We additionally compare causal steering responses with associations observed under unedited validation inputs. For each feature and statistic, the natural response is the Pearson correlation between feature activation and the statistic over all validation scenes. The steering response is the standardized effect E_{jm} produced by the positive intervention. We report results over the 270 features that activate in at least 64 validation scenes.

Statistic	Sign match	Pearson	Spearman
Lateral displacement	59.7%	0.686	0.349
Curvature	40.3%	−0.289	−0.025
Brake magnitude	56.7%	0.434	0.368
Score margin	58.1%	0.233	0.283
Proposal spread	61.5%	0.231	0.352

Table 8: Agreement between naturally occurring activation and output associations and positive intervention responses. Pearson and Spearman columns measure the relationship across features between natural association strength and the effect of intervening on SAE features.

Natural associations predict intervention direction most strongly for lateral displacement. Braking, score margin, and proposal spread also show positive, but moderate, agreement. Curvature is the main exception, consistent with its greater numerical sensitivity. The exhaustive sweep evaluates only positive latent edits because the top-k SAE latents are nonnegative; we therefore do not infer symmetric negative-edit behavior from these results.

G Scorer Adapter Details and Ablations

We train a lightweight *scorer adapter* that corrects the frozen planner's proposal-ranking scores using internal planner representations. The adapter does not change trajectory generation: it reranks the same $K{=}50$ proposals produced by the base model by adding offsets to the generated scores. All results are evaluated on the entire WOD-E2E validation split, and metrics are computed over all 50 proposals per prediction. **ADE@5s** is the ADE of the scorer's top-1 pick (lower is better). **Mean rank** is the oracle rank (1 = best) of the selected proposal, averaged over scenes (lower is better). **Relative improvement** is $(\mathrm{ADE}_{\mathrm{base}} - \mathrm{ADE}_{\mathrm{method}})/\mathrm{ADE}_{\mathrm{base}} \times 100\%$.

G.1 Adapter Architecture

Setup. Let the frozen planner produce, for each scene i, a set of $K{=}50$ proposal trajectories $\{\boldsymbol{\tau}_{ik}\}_{k=1}^{K}$ with $\boldsymbol{\tau}_{ik} \in \mathbb{R}^{2T}$, $T{=}20$ (5 s horizon at $\Delta t{=}0.25$ s), and corresponding base scorer predictions $\hat{s}_{ik} = S_\theta(\boldsymbol{\tau}_{ik}, q_i)$, where q_i is the final query token. The true per-proposal ADE is $a_{ik} = \mathrm{ADE}(\boldsymbol{\tau}_{ik}, \mathbf{y}_i)$. We hook the planner at block $b{=}3$ and capture the dense activation $h_i \in \mathbb{R}^d$ with $d{=}384$. Our frozen SAE maps this activation to sparse latents using the same preprocessing and TopK encoder as in all other experiments:

$$z_i = \mathrm{ReLU}\Big(\mathrm{TopK}(W_{\mathrm{enc}}\tilde{h}_i, k)\Big) \in \mathbb{R}^M, \tag{23}$$

where $\tilde{h}_i$ is the preprocessed activation obtained by subtracting the learned preprocessing bias, removing the per-sample mean across feature dimensions, and normalizing to unit L_2 norm, as described in the implementation details.

Adapter. For each proposal (i, k), the adapter f_ϕ takes three inputs:

1. the SAE latent z_i (scene-level internal representation),
2. the flattened proposal $\boldsymbol{\tau}_{ik} \in \mathbb{R}^{40}$,
3. the base score $\hat{s}_{ik} \in \mathbb{R}$.

We embed each input into a hidden dimension of 256 and pass them through a two-layer MLP head to predict a scalar *scorer residual*

$$\hat{r}_{ik} = f_\phi(z_i,\, \boldsymbol{\tau}_{ik},\, \hat{s}_{ik}). \tag{24}$$

The corrected score is

$$\tilde{s}_{ik} = \hat{s}_{ik} + \hat{r}_{ik}, \tag{25}$$

and the adapter selects $\arg\min_k \tilde{s}_{ik}$. The planner backbone and SAE are frozen throughout; only ϕ is trained.

Training objective. The regression target is the true scorer error $r_{ik} = a_{ik} - \hat{s}_{ik}$. We minimize an L_1 loss over proposals:

$$\mathcal{L}_{\mathrm{reg}} = \frac{1}{BK} \sum_{i,k} w_{ik} \left|\hat{r}_{ik} - r_{ik}\right|, \tag{26}$$

where w_{ik} is 3.0 to the top 10 proposals ranked by ground truth ADE, and is 1.0 for the other 40 proposals.

Hyperparameters. We train the adapter for 8 epochs using AdamW with learning rate 10^{-3}, weight decay 10^{-3}, batch size 16, and dropout 0.2, with early stopping on validation loss.

G.2 Dense Query Control

To test whether sparsity is necessary, we train an ablation that replaces the SAE latent z_i with the raw dense hooked activation $h_i \in \mathbb{R}^{384}$ from the same block-3 MLP output. All other components are identical: same three-tower fusion head, same trajectory and base score inputs, same ℓ_1 residual target, same frozen backbone, and same training schedule. The only difference is the input: replacing the SAE latent with the original query. This control isolates the value of SAE disentanglement: if sparse latents merely provide extra capacity, the dense adapter should match SAE-latent performance; if SAE features expose task-relevant structure, the SAE-latent adapter should outperform the dense control.

G.3 Matched Trainable Control

To further test whether the SAE adapter benefits simply from the additional encoder parameters, we replace the pretrained frozen SAE encoder with a randomly initialized encoder of identical architecture and parameter count. We train this encoder jointly with the adapter head on the same L_1 scorer-residual objective while keeping the base planner frozen. Thus, the two variants differ only in whether the encoder representation is learned through SAE pretraining or directly from the downstream score-correction objective. This randomly initialized control reduces ADE@5s by 2.86%, compared with 3.44% for the frozen SAE encoder.

G.4 Results

Table 9 summarizes the ablation. The SAE-latent adapter improves mean rank from 16.7 to 15.6 and reduces ADE@5s by 3.44% relative to the base scorer. The dense-query control improves over the base scorer but falls short of the SAE-latent variant, indicating that sparse SAE features carry information useful for scoring beyond what is available in the raw dense activation. The randomly initialized adapter matching the SAE's architecture also does not produce as large of an improvement over the base scorer as the original SAE latent adapter.

These results provide evidence that SAE latents expose internal structure (such as object- and intent-correlated features identified in Section E) that can be utilized to improve model performance after training without requiring supervised fine-tuning of the entire model.

Method	Selected mean rank ↓	ADE@5s reduction ↑
Base scorer	16.7	–
Dense query adapter	16.2	2.74%
Random init encoder adapter	16.1	2.86%
SAE latent adapter	15.6	**3.44%**

Table 9: Scorer adapter ablations on WOD-E2E validation. ADE values report relative reduction from the base scorer. The dense-query control replaces SAE latents with the original block-3 activation, while the random-init control replaces the pretrained SAE encoder with an encoder of matched architecture and parameter count trained on the downstream objective.